

La política de control DVFS del daemon — explicación y justificación detallada

Proyecto: Hyperion — agente en espacio de usuario para gestión dinámica de frecuencia en sistemas heterogéneos CPU–GPU **Nodo objetivo:** **paccaA100** (Intel Xeon Gold 5317, Ice Lake-SP; NVIDIA A100-PCIe-40GB) **Fecha:** 18 de agosto de 2026

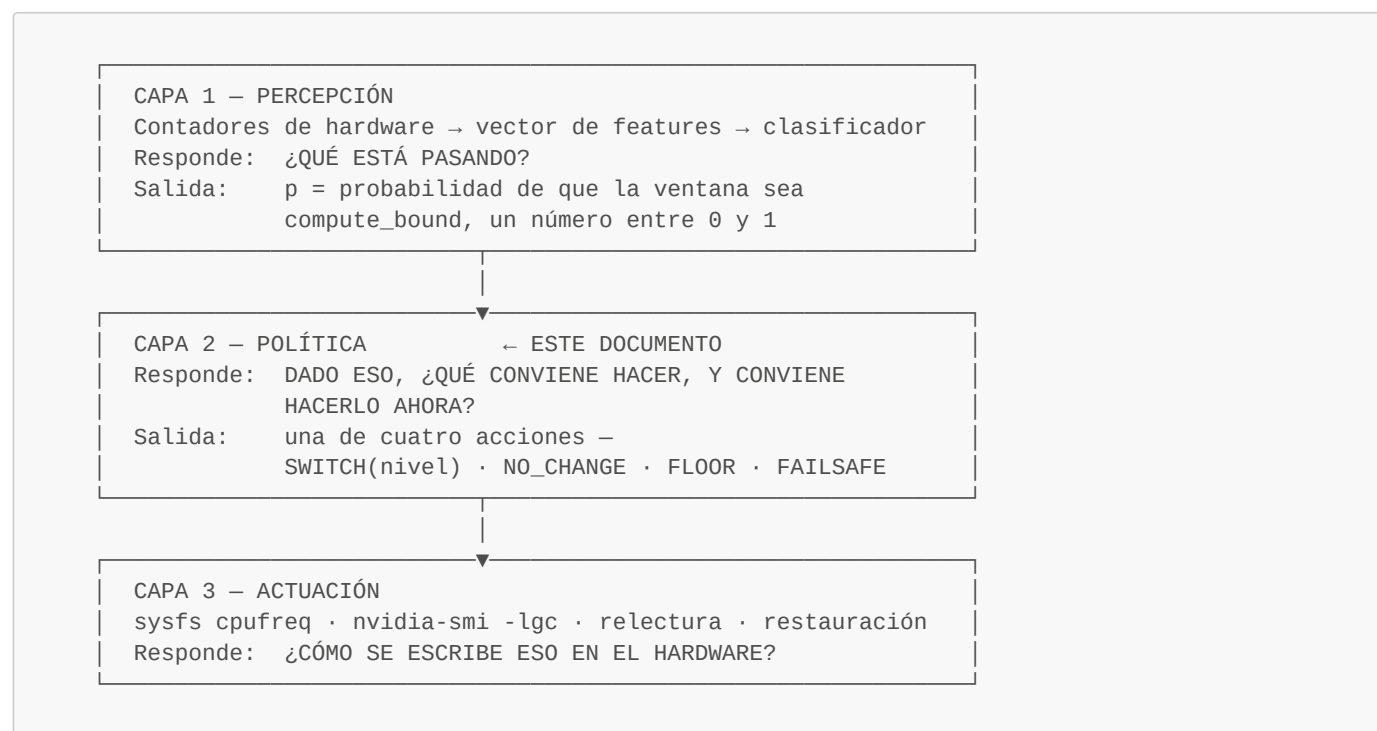
Este documento explica **una sola cosa**: qué es la política de control del daemon, de qué partes está hecha, qué hace cada parte, por qué existe, qué valor tiene cada variable, de dónde sale ese valor y qué pasa si se cambia. Cada mecanismo se acompaña de un ejemplo numérico trabajado.

No describe el flujo de arranque, la arquitectura del proceso ni el plan de construcción por etapas — eso está en el plan de implementación de la Fase 3. Aquí solo está la política.

Advertencia sobre los números. Los valores marcados **[medido]** provienen de campañas reales de este proyecto en **paccaA100**. Los marcados **[ilustrativo]** son ejemplos contruidos con magnitudes plausibles para explicar un mecanismo; **no son resultados** y no deben citarse como tales. Los valores finales de la tabla de frecuencias solo pueden salir de la campaña de caracterización multi-frecuencia, que todavía no se ha ejecutado.

1. Qué es exactamente "la política"

La palabra *política* se usa con demasiada holgura, así que conviene fijarla. En Hyperion hay **tres capas separadas**, y solo la del medio es la política:

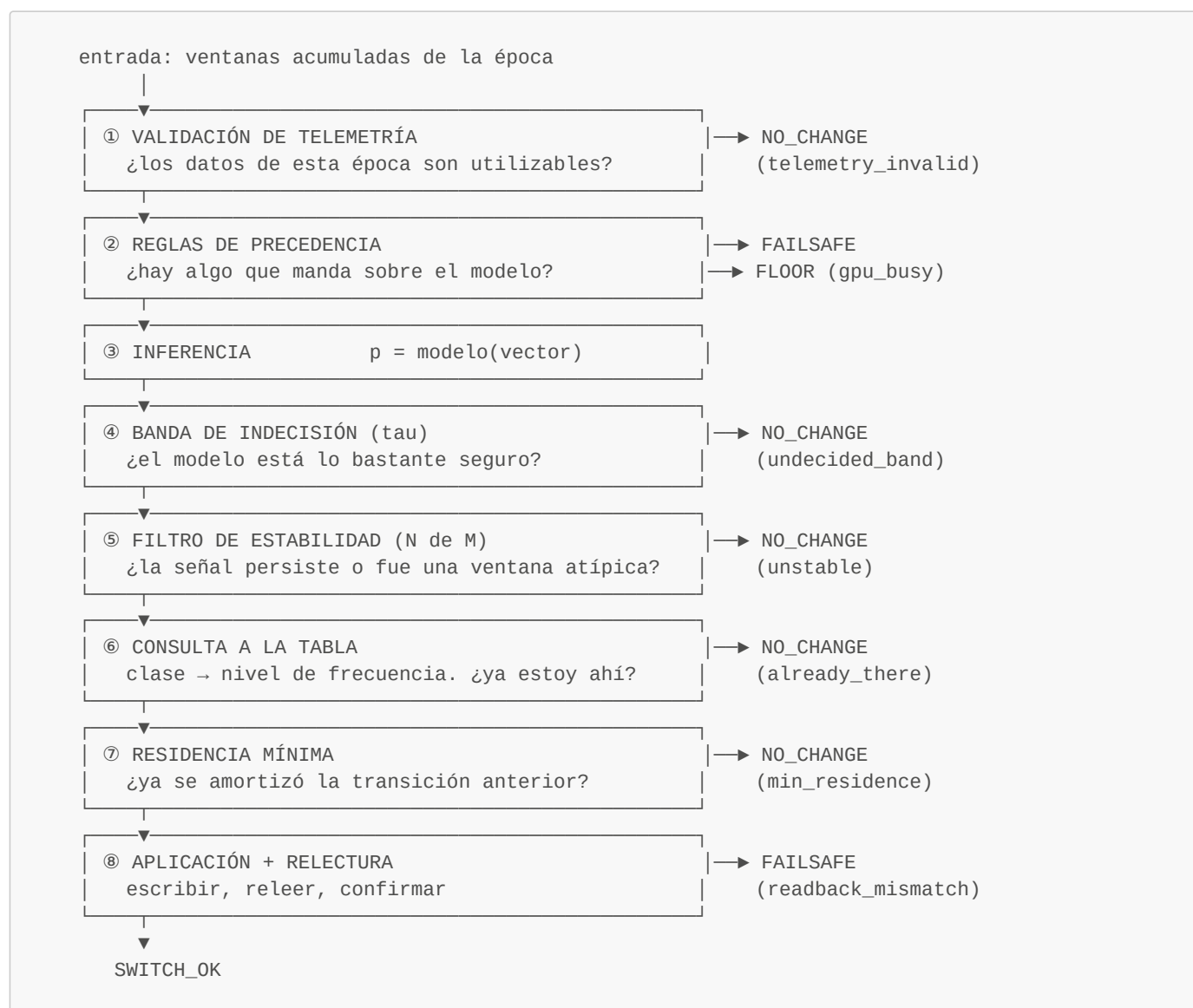


Por qué importa la separación. Si el clasificador escogiera frecuencia directamente, las tres capas colapsarían en una y se perderían tres cosas: (a) la posibilidad de saber, cuando el resultado decepcione, cuál de las tres falló; (b) la posibilidad de construir y probar la política antes de que el modelo exista; y (c) la compatibilidad con el plan de trabajo de grado aprobado, que fija explícitamente que la salida del modelo es una etiqueta de régimen, no una frecuencia.

La política es determinista. Dado el mismo p , el mismo estado interno y la misma configuración, produce siempre la misma acción. No hay aleatoriedad, no hay exploración, no hay aprendizaje en línea. Esto es lo que hace que cada decisión se pueda reconstruir a posteriori desde el log, que es la propiedad de interpretabilidad que se defiende en la sustentación.

2. Anatomía: los ocho componentes, en su orden de evaluación

La política es una secuencia de ocho componentes que se evalúan **siempre en este orden**. El orden no es arbitrario: cada componente es más caro o más comprometedor que el anterior, así que se colocan de barato-y-seguro a caro-y-comprometedor. Cinco de los ocho pueden terminar la decisión sin actuar.



La asimetría es deliberada: llegar hasta ⑧ requiere superar siete puertas; quedarse quieto requiere que falle una sola. En un nodo compartido donde cada actuación cuesta tiempo, energía y riesgo para otros usuarios, esa es la asimetría correcta.

3. Componente por componente

① Validación de telemetría

Qué es. Una comprobación de que las métricas de esta época se pueden usar. Tres condiciones deben cumplirse:

```

running_ratio = time_running / time_enabled ≥ 0.95
delta_cycles > 0      (denominador de ipc y stall_ratio)
delta_instructions > 0 (denominador de mpki)
  
```

Por qué existe. El presupuesto de contadores físicos simultáneos de la microarquitectura es limitado. Si se piden más eventos de los que caben, el kernel de Linux los rota en el tiempo y **escala los valores por extrapolación**. Un **ipc** calculado sobre contadores multiplexados no es un **ipc** medido: es una estimación con un error que depende de qué tan sincronizada estuvo la rotación con la fase de la aplicación. Inferir sobre eso es inferir sobre ruido con apariencia de dato.

La segunda condición viene de una regla que en este proyecto es absoluta: **nunca dividir sin verificar el denominador**. Un denominador cero produce un valor no numérico explícito y un estado de calidad — nunca un cero silencioso, nunca una excepción no controlada. Esta regla salió de una lección real: los errores de datos en este pipeline no se manifiestan como fallos ruidosos, se manifiestan como resultados plausibles y equivocados. La campaña donde `windows.csv` salía vacío en 14 de 21 corridas pasó una revisión superficial de "21/21 aceptadas" sin que nadie lo notara.

Ejemplo. Una época con `time_enabled = 10 ms` pero `time_running = 6,2 ms` da `running_ratio = 0,62`: los contadores solo estuvieron activos el 62 % del intervalo y los conteos vienen multiplicados por $1/0,62$. La política devuelve `NO_CHANGE` con motivo `telemetry_invalid` y **no llama al clasificador**. Si en cambio se hubiera inferido, un `mpki` real de 18 se habría presentado como 29 y la ventana habría cruzado de clase por un artefacto de medición.

Si se quitara este componente: el daemon tomaría decisiones sobre datos extrapolados en las épocas donde el sistema esté bajo presión de contadores — que son precisamente las épocas de arranque de fase, las más importantes.

② Reglas de precedencia

Qué es. Tres condiciones que, si se cumplen, deciden la acción **sin consultar al modelo**. Se evalúan antes de la inferencia, en este orden:

Prioridad	Condición	Acción
P-1	Hardware insano: temperatura fuera de rango, indicación de <i>throttling</i> , núcleo delegado que no responde	<code>FAILSAFE</code> + restaurar
P-2	<code>gpu_busy == true</code>	CPU al piso, sin inferencia
P-3	(ya cubierta por ①) telemetría inválida	<code>NO_CHANGE</code>

Por qué existe P-2, que es la interesante. Cuando el CPU espera a que la GPU termine, con el comportamiento por defecto de CUDA, el hilo de CPU **no se bloquea**: ejecuta un bucle de espera activa. Desde los contadores eso se ve así:

Métrica	Espera activa por GPU	Carga compute-bound real
<code>ipc</code>	alto (~1,5–2,0)	alto (~1,5–2,0)
<code>mpki</code>	casi cero	bajo
<code>stall_backend_ratio</code>	bajo	bajo

Son **indistinguibles**. Un clasificador correcto y bien entrenado diría `compute_bound`, y la política subiría la frecuencia justo cuando el CPU no está haciendo ningún trabajo útil. No es un fallo del modelo: es un caso donde la señal es genuinamente engañosa, y ninguna cantidad de entrenamiento lo arregla porque los vectores son iguales.

Este proyecto ya midió el efecto: con una biblioteca precargada que fuerza la espera bloqueante en binarios de terceros, el uso de CPU durante la espera pasó de **99,8 % a 0,0 %** [medido], sin alterar la salida de los kernels. Pero ese mecanismo solo aplica a las cargas del catálogo; una aplicación arbitraria en producción no lo tiene, y por eso la regla vive en la política.

Y en el caso contrario —espera realmente bloqueante— la regla se aplica igual. Cuando el hilo sale de la cola de ejecución, el estado de frecuencia casi no afecta el consumo (la potencia dinámica requiere conmutación de compuertas, y un hilo bloqueado no ejecuta nada). Forzar el piso no ayuda, pero tampoco cuesta; y como muchas implementaciones de espera bloqueante hacen un giro corto antes de bloquearse de verdad, cubrir ese margen es gratis. En un caso la regla **corrige un error**, en el otro es **una defensa sin costo**.

La señal es unidireccional: el ciclo de GPU la emite, el de CPU la consume, y el de GPU nunca lee nada del de CPU. Esto evita cualquier negociación entre dominios, que sería otro proyecto.

③ Inferencia

Qué es. La única llamada al modelo por época:

```
p = clasificador.predict_proba(vector)[compute_bound]
```

$p \in [0, 1]$ es la probabilidad estimada de que la época esté en régimen limitado por cómputo.

Por qué la probabilidad y no la etiqueta. Una etiqueta dura (`compute_bound` / `memory_bound`, sin más) descarta la única información que distingue dos situaciones muy distintas: una ventana que es inequívocamente de un régimen, y una que quedó del lado correcto de la frontera por muy poco. Con solo la etiqueta, ambas se ven idénticas para la política — y son exactamente las segundas las que producen conmutación sin beneficio real: dos ventanas consecutivas con probabilidades 0,51 y 0,49 caen en clases opuestas aunque el sistema, en la práctica, no cambió de régimen. Sin una noción de "qué tan compute o qué tan memory", la política no tiene manera de distinguir ese caso de un cambio de fase genuino, y conmuta contra sí misma.

Esta decisión ya estaba tomada en el diseño de la política (banda de indecisión, componente ④) y aquí queda fijada como requisito explícito hacia la Fase 2: **el clasificador debe exponer `predict_proba`, no solo `predict`**. No es una ampliación de alcance. Un árbol de decisión devuelve la proporción de muestras de entrenamiento de cada clase en la hoja alcanzada; un bosque aleatorio devuelve la fracción de árboles que votaron cada clase — son exactamente las dos familias de modelo que el plan de trabajo de grado nombra como candidatas, y ambas exponen esa fracción en su implementación estándar (`scikit-learn`) sin entrenamiento adicional, sin hiperparámetros nuevos y sin costo de inferencia extra: es información que el modelo ya calculó para decidir la etiqueta, expuesta en vez de descartada.

Esto no contradice el plan aprobado. El plan (§5.2) dice que el modelo "produce como salida una etiqueta que representa la fase de ejecución" — y eso se sigue cumpliendo: `etiqueta = argmax(predict_proba)`. Fase 2 se evalúa y se reporta exactamente como el plan describe (una tarea de clasificación con dos clases, medida con métricas de clasificación estándar). Lo único que cambia es que el artefacto serializado que Fase 2 entrega a Fase 3 expone, además de la etiqueta, la probabilidad que ya calculó para llegar a ella — un detalle del formato del artefacto, no un cambio del problema que Fase 2 resuelve.

Contrato con la Fase 2. El modelo entra por aquí y por ningún otro lugar. La política no sabe ni le importa si dentro hay un árbol o un bosque. Esto es lo que permite sustituirlo por tres implementaciones alternativas que respetan el mismo contrato:

- **clasificador de traza:** devuelve las etiquetas ya calculadas de una campaña anterior; permite validar la política completa contra datos reales sin ejecutar nada;
- **clasificador oráculo:** devuelve la etiqueta verdadera del Roofline; permite medir el techo de lo que la política puede lograr si la clasificación fuera perfecta — es uno de los cinco tratamientos experimentales;
- **clasificador de prueba:** devuelve secuencias fijas; permite ejercitar cada rama de la política de forma determinista.

④ Banda de indecisión

Qué es. Una zona muerta alrededor de la frontera de decisión:

```
p ≥ 0,5 + tau → candidato HIGH
p ≤ 0,5 - tau → candidato LOW
en otro caso  → NO_CHANGE
```

Con el valor por defecto `tau = 0,15`, la banda muerta es $p \in (0,35 ; 0,65)$.

Por qué existe. Porque hay cargas cuyo régimen **genuinamente no está definido** a la resolución de una época, y para esas cargas una decisión binaria dura conmuta permanentemente por diseño. Esto no es una hipótesis: está en los datos de este proyecto.

Kernel	Ventanas <code>memory_bound</code> clase B	Clase C	Lectura
<code>npb_mg</code>	99,9 %	99,9 %	inequívoco
<code>npb_cg</code>	92,7 %	93,5 %	inequívoco
<code>npb_bt</code>	14,4 %	14,6 %	inequívoco (cómputo)
<code>npb_lu</code>	11,6 %	11,0 %	inequívoco (cómputo)
<code>npb_sp</code>	58,2 %	59,3 %	mixto

Kernel	Ventanas <code>memory_bound</code>	clase B	Clase C	Lectura
<code>npb_ft</code>		20,3 %	33,8 %	cambia con el tamaño

[medido, campañas de `paccaA100`, 1 107 573 ventanas]

`npb_sp` reparte sus ventanas casi mitad y mitad, de forma **estable entre dos tamaños de problema distintos**. No es ruido de medición: es una carga cuyo comportamiento alterna a una granularidad más fina que la época. Sin banda de indecisión, `npb_sp` produciría un `p` que cruza 0,5 constantemente y la política intentaría conmutar sin parar.

NO_CHANGE no significa "no sé qué hacer": **significa que quedarse es la decisión correcta**. Mantener el estado actual evita pagar transiciones por una señal que no las justifica.

Ejemplo numérico. Tres épocas consecutivas con `tau = 0,15`:

Época	<code>p</code>	Con <code>tau = 0,15</code>	Con <code>tau = 0,05</code>	Con <code>tau = 0,30</code>
11	0,52	NO_CHANGE	candidato HIGH	NO_CHANGE
12	0,47	NO_CHANGE	candidato LOW	NO_CHANGE
13	0,54	NO_CHANGE	candidato HIGH	NO_CHANGE

Con `tau = 0,05` esas tres épocas producen la secuencia **HIGH → LOW → HIGH**: dos transiciones candidatas en 30 ms, sobre un modelo que en ningún momento superó el 54 % de confianza. Con `tau = 0,30` la banda se vuelve tan ancha que también cargas razonablemente claras (`p = 0,72`) quedarían atrapadas, y el daemon degeneraría en frecuencia fija.

Cómo se calibra `tau`. No se elige a ojo. Se barre `tau ∈ {0,05 · 0,10 · 0,15 · 0,20 · 0,25}` sobre el conjunto de validación de la Fase 2 y se elige el valor que minimiza el número de transiciones sin degradar el EDP en la reproducción sobre trazas. Es un experimento **completamente offline**: no consume tiempo de nodo ni requiere ningún permiso.

⑤ Filtro de estabilidad `N` de `M`

Qué es. El candidato debe repetirse en al menos `N` de las últimas `M` épocas para ser aceptado. Por defecto **3 de 4**.

Por qué existe, y por qué es distinto de ④. Los dos componentes filtran cosas diferentes y son complementarios:

- **La banda de indecisión filtra incertidumbre del modelo.** Actúa sobre *una* época: si el modelo no está seguro, no se decide. Atrapa cargas mixtas, que producen `p ≈ 0,5` de forma sostenida.
- **El filtro `N` de `M` filtra ruido temporal.** Actúa sobre la *secuencia*: si la señal no persiste, no se decide. Atrapa ventanas atípicas aisladas — una interrupción del sistema, una migración de hilo, un fallo de página — que producen un `p` extremo durante una sola época.

Una carga mixta pasa el filtro temporal (su señal es persistente) pero la atrapa la banda. Una ventana atípica pasa la banda (`p = 0,08`, altísima confianza) pero la atrapa el filtro. Se necesitan los dos.

Por qué 3 de 4 y no 3 consecutivas. Un criterio de consecutividad estricta se reinicia por completo ante una sola época discrepante: la secuencia **L L L H L L L** nunca alcanzaría tres consecutivas si la **H** cae en el peor lugar. El criterio **N de M** tolera esa discrepancia y sigue reconociendo una fase que es claramente de memoria. Es más robusto por el mismo costo computacional.

El costo: latencia de detección.

```
latencia_mínima = N × duración_de_época = 3 × 10 ms = 30 ms
latencia_máxima = M × duración_de_época = 4 × 10 ms = 40 ms
```

Ejemplo. Con historial **[HIGH, HIGH, HIGH, LOW]** y candidato **LOW**, el conteo de **LOW** en las últimas 4 épocas es 1: no llega a 3, se devuelve **NO_CHANGE** con motivo **unstable**. Ese único **LOW** era una ventana atípica y el filtro hizo su trabajo. Tres épocas después, con historial **[HIGH, LOW, LOW, LOW]**, el conteo es 3: la fase es real y se acepta.

Qué pasa si se cambia. Con **N = 1** (sin filtro), cada ventana atípica produce una transición candidata. Con **N = 8 de 10**, la latencia de detección sube a 80–100 ms y se suma a la residencia mínima, de modo que solo fases muy largas serían aprovechables.

⑥ Consulta a la tabla

Qué es. Un diccionario de dos entradas por dominio, consultado en tiempo constante:

```
policy["cpu"]["compute_bound"] → HIGH → un nivel de frecuencia concreto
policy["cpu"]["memory_bound"] → LOW → un nivel de frecuencia concreto
```

Si el nivel resultante coincide con el vigente, se devuelve **NO_CHANGE** con motivo **already_there**.

Por qué una tabla y no un cálculo. Porque el cálculo correcto —¿qué frecuencia minimiza el EDP de esta ventana?— **no se puede hacer en línea, ni siquiera en principio**. El EDP de una ventana ejecutada a una frecuencia no dice cuál habría sido el EDP simultáneo de esa misma ventana a otra frecuencia: la ventana ya pasó y no se puede repetir. Descubrirlo exigiría probar frecuencias durante la ejecución real, lo que contamina la aplicación que se está midiendo y multiplica las transiciones.

Por eso el proyecto separa los tiempos:

```
OFFLINE (campana):    energía + tiempo medidos a F0...F4 → seleccionar HIGH y LOW
EN LÍNEA (daemon):    clase → consultar tabla [0(1)]
POSTERIOR (Fase 4):    energía + tiempo medidos → verificar si mejoró el EDP
```

Por qué dos estados y no cinco. La campaña explora cinco niveles **F0...F4** porque se necesita **caracterizar la curva completa** para el capítulo de resultados. Pero la **decisión** usa solo dos, por dos razones:

1. *Razón lógica:* un clasificador binario no puede responder "**memory_bound** → ¿F1, F2, F3 o F4?". Resolver eso exigiría un segundo predictor o una heurística, es decir, otro proyecto.
2. *Razón estadística, que es la más fuerte:* con 6–7 kernels de dataset y 3 repeticiones, cada celda (**clase**, **nivel**) tiene como máximo 3 observaciones. Una selección entre cinco niveles sería indefendible con ese **n**. Con dos estados efectivos, la decisión recae sobre una comparación binaria, que sí se sostiene.

La tabla es un artefacto congelado con suma de verificación. El daemon se niega a arrancar si la suma de verificación de la calibración registrada en la política no coincide con la del archivo de calibración presente: una tabla calibrada contra otro Roofline contiene números que no significan lo que dicen.

⑦ Residencia mínima

Qué es. Un tiempo mínimo que una frecuencia aplicada debe permanecer antes de poder cambiar, sin importar lo que pidan las épocas siguientes. Por defecto **100 ms en CPU** y **1000 ms en GPU**.

Por qué existe. Porque cambiar de frecuencia **cuesta**. El costo tiene dos componentes: el tiempo durante el cual el procesador no ejecuta a ninguna de las dos frecuencias, y la energía de la propia transición. Las magnitudes documentadas:

Fuente	Latencia de transición
Plan de trabajo de grado, §4.1.6	~10 ms sin control por hardware; ~1 ms con él
Carpentieri et al., IPDPS 2025	0,33 ms (AMD MI100) · 0,30 ms (Intel Max 1100) · 0,60 ms (NVIDIA V100S)
Veličká et al., IPDPSW 2025	en GPUs modernas depende del par origen–destino; hasta decenas de milisegundos

De dónde sale el valor 100 ms. De una regla de amortización explícita: se exige que el costo de transición no supere el 10 % del tiempo que se pasa en el estado alcanzado.

```
fracción_de_overhead = T_transición / T_residencia ≤ 10 %
⇒ T_residencia ≥ 10 × T_transición
⇒ T_residencia ≥ 10 × 10 ms = 100 ms
```

Se usa el valor pesimista de 10 ms porque no se ha medido la latencia real en `paccaA100` — y no se puede medir sin el permiso de escritura de frecuencia. **Ese es el número que debe reemplazar al valor por defecto en cuanto el permiso llegue:** la residencia mínima se recalibra contra la latencia medida en el nodo, no contra un valor de literatura.

Conceptualmente, el propio Linux respalda el razonamiento: el governor `schedutil` impone un `rate_limit_us` que por defecto se deriva de la latencia de transición del driver. La idea de fondo es la misma — ninguna política debe actuar más rápido de lo que el hardware puede materializar el cambio.

La consecuencia honesta, que hay que declarar. Sumando la latencia de detección del componente ⑤ y la residencia:

tiempo mínimo para reaccionar a una fase	=	30–40 ms	(detección)
tiempo mínimo en el estado alcanzado	=	100 ms	(residencia)
duración mínima de fase aprovechable	≈	140 ms	

Una fase más corta que ~140 ms no puede ser explotada por esta política. No es un defecto oculto: es el precio de no gastar más en conmutar de lo que se ahorra, y debe aparecer explícitamente en el capítulo de limitaciones. Si la evaluación experimental muestra que la mejor frecuencia estática le gana al oráculo, una de las explicaciones candidatas es precisamente que las fases de esas cargas son más cortas que este umbral.

Ejemplo. El daemon aplicó `F1` en $t = 0$. En $t = 70$ ms la política ha detectado de forma estable un régimen de memoria y quiere aplicar `F2`. La residencia transcurrida es $70\text{ ms} < 100\text{ ms}$: se devuelve `NO_CHANGE` con motivo `min_residence` y `residence_remaining = 30 ms`. El cambio se aplica en la primera época a partir de $t = 100$ ms.

Qué pasa si se cambia. Con residencia de 20 ms, una carga que alterne fases de 50 ms conmutaría continuamente y pagaría 10 ms de transición por cada 50 ms de ejecución: un 20 % de sobre costo, que probablemente excede todo el ahorro energético disponible. Con residencia de 1000 ms, el daemon se pierde casi todas las fases reales de las cargas del catálogo y converge en la práctica a una frecuencia fija — es decir, se vuelve indistinguible del baseline contra el que debe demostrar mejora.

⑧ Aplicación y relectura

Qué es. Escribir la frecuencia por la interfaz del sistema y **volver a leerla desde el hardware** para confirmar que se aplicó.

```
apply_frequency(dominio, objetivo)
observado = read_frequency(dominio)          # scaling_cur_freq / clocks.sm
si observado no es compatible con objetivo:
    restaurar_estado_original(dominio)
    FAILSAFE
```

Por qué la relectura es obligatoria. Porque una escritura al sysfs que no lanza error **no significa que la frecuencia haya cambiado**. El kernel puede aceptar la escritura y luego el hardware ignorarla por límites térmicos, por un límite de potencia, por interacción con el control de estados de rendimiento por hardware, o porque el valor pedido no está entre los soportados. En este proyecto la regla es un no-negociable: *toda escritura de frecuencia se verifica por relectura; nunca se asume éxito porque la llamada no lanzó excepción*.

Por qué FAILSAFE no reintenta. Un actuador que no responde como se le pide es un actuador que no se entiende. Seguir escribiéndole en un nodo compartido con otros usuarios es exactamente lo que no se debe hacer. `FAILSAFE` significa: restaurar el estado original y **dejar de actuar** — no reintentar, no ajustar, no "corregir".

Nota sobre la relectura en GPU. El código del proyecto ya hace esto con una consulta independiente (`nvidia-smi --query-gpu=clocks.sm`) en lugar de confiar en el código de retorno de `-lgc`. Y hay un matiz documentado: el reloj observado puede legítimamente no coincidir con el pedido si la GPU está en reposo, porque el bloqueo fija un techo, no un piso. Eso es comportamiento esperado, no una falla; lo que sí es falla es un reloj observado por encima del bloqueo pedido.

4. Las variables, una por una

4.1 La distinción *oracle* / *trace*

Es la distinción conceptual central y conviene enunciarla con precisión:

- **Variable oracle:** su valor proviene de conocimiento adquirido **antes** de la ejecución — de la campaña de caracterización, de la calibración del nodo, o del entrenamiento del modelo. Durante la ejecución es **constante y de solo lectura**.
- **Variable trace:** se **observa del sistema en tiempo real**, época a época. El daemon nunca la fija; solo la lee.

La separación no es organizativa sino epistemológica: **las variables oracle son las hipótesis del sistema; las variables trace son la evidencia**. Cada decisión de la política es una confrontación entre unas y otras.

Y de ahí sale una regla dura: **ninguna variable oracle puede cambiar durante una ejecución**. Si **tau** o la tabla se modificaran en caliente, la política dejaría de ser la que se calibró y validó, y ningún resultado experimental sería atribuible a nada. El daemon carga los artefactos al arrancar, verifica sus sumas de verificación, y no vuelve a tocarlos.

4.2 Variables oracle

i_ridge_ref — el punto de inflexión del modelo Roofline, congelado

- **Qué representa físicamente:** la intensidad operacional, en FLOP por byte, donde el techo de cómputo del nodo se cruza con el techo de memoria. Por debajo, una carga no puede alimentar a las unidades aritméticas lo bastante rápido y está limitada por memoria; por encima, la memoria alcanza y el límite es aritmético.
 - **De dónde sale:** $i_ridge = P_pico / BW_pico$, ambos **medidos en el propio nodo** — STREAM y ERT en CPU; BabelStream y un microbenchmark propio en GPU.
 - **Valores medidos en paccaA100 [medido]:** GPU, precisión simple: $10\ 178,2\ GFLOP/s \div 1\ 399\ GB/s = 7,28\ FLOP/byte$. GPU, doble precisión: $4\ 698,6 \div 1\ 399 = 3,36\ FLOP/byte$. El valor de CPU sale de la calibración de sesión.
 - **Por qué está congelado (lo más importante de esta variable):** porque **P_pico escala con el reloj** y **BW_pico** casi no lo hace — el reloj de memoria pertenece a otro dominio que este proyecto no toca. Si la etiqueta se calculara contra el ridge del estado *actual*, bajar la frecuencia bajaría el ridge, lo que empujaría a la misma carga hacia el lado de cómputo, lo que subiría la frecuencia. **Es un lazo cerrado**. La sección 8.2 lo desarrolla con números reales.
 - **Si es demasiado alto:** cargas realmente limitadas por cómputo se clasifican como de memoria y se les baja la frecuencia — pérdida de rendimiento sin ahorro proporcional.
 - **Si es demasiado bajo:** cargas de memoria corren a frecuencia alta, desperdiciando energía en ciclos de espera.
-

HIGH[dominio] y LOW[dominio] — los dos estados efectivos

- **Qué representan:** los dos únicos niveles de frecuencia que la política puede aplicar en un dominio. **No son "máximo" y "mínimo" por definición:** son los que minimizan el EDP mediano bajo restricción de degradación.
 - **De dónde salen:** de la campaña de caracterización multi-frecuencia. Sección 5 de este documento, con ejemplo completo.
 - **Valor actual:** **null**. La campaña no se ha ejecutado (bloqueada por permisos). **Un valor inventado aquí invalidaría todo resultado posterior**, así que el archivo de política lleva nulos explícitos hasta que existan los números reales.
 - **Si quedan muy juntos:** la política es indistinguible de una frecuencia fija y el trabajo no puede demostrar valor.
 - **Si quedan muy separados:** cada transición cuesta más (la latencia depende del par origen–destino) y el riesgo de degradación de rendimiento crece.
-

tau — media anchura de la banda de indecisión

- **Qué representa:** cuánta confianza se le exige al modelo antes de aceptar su clase. Es el precio de admisión de una predicción.
 - **Valor por defecto:** **0,15** → se exige $p \geq 0,65$ o $p \leq 0,35$.
 - **De dónde sale:** barrido offline sobre $\{0,05 \dots 0,25\}$ en el conjunto de validación de la Fase 2, eligiendo el valor que minimiza transiciones sin degradar el EDP en reproducción sobre trazas.
 - **Ejemplo:** ver la tabla comparativa del componente ④ — con **tau** = **0,05**, tres épocas con **p** de 0,52 / 0,47 / 0,54 generan dos transiciones candidatas; con **tau** = **0,15**, ninguna.
 - **Si es demasiado alto:** el daemon casi nunca decide y degenera en frecuencia fija.
 - **Si es demasiado bajo:** actúa sobre predicciones de baja confianza y multiplica las transiciones inútiles, justo en las cargas mixtas donde más daño hace.
-

n_of_m — filtro de estabilidad

- **Qué representa:** cuántas de las últimas **M** épocas deben coincidir para aceptar un cambio de clase.

- **Valor por defecto: 3 de 4.**
- **De dónde sale:** compromiso entre latencia de detección (30–40 ms con épocas de 10 ms) y rechazo de ventanas atípicas. Se calibra junto con **tau** sobre trazas reales.
- **Si es demasiado alto:** detección lenta; las fases cortas terminan antes de que el daemon reaccione.
- **Si es demasiado bajo:** cada ventana atípica produce una transición candidata.

min_residence_ms — barrera de amortización

- **Qué representa:** el tiempo mínimo que una frecuencia aplicada debe permanecer. Es lo que impide que el costo de transición se coma el ahorro.
- **Valor por defecto: 100 ms (CPU), 1000 ms (GPU).**
- **De dónde sale:** $T_{residencia} \geq 10 \times T_{transición}$, con $T_{transición} = 10$ ms (valor pesimista del plan aprobado). **Debe recalibrarse contra la latencia medida** en cuanto llegue el permiso de control de frecuencia.
- **Si es demasiado alto:** el daemon se pierde fases legítimas y converge a frecuencia fija.
- **Si es demasiado bajo:** el sobrecosto de conmutación anula el ahorro. Con residencia de 20 ms y fases de 50 ms, el 20 % del tiempo se va en transicionar.

slowdown_limit — restricción de rendimiento

- **Qué representa:** la degradación relativa de tiempo de ejecución máxima aceptable frente al nivel de referencia. Es la restricción bajo la cual se minimiza el EDP.
- **Valor por defecto: 5 %**, con 3 % y 10 % estudiados en el piloto y el valor **congelado antes** de la campaña final.
- **Por qué existe:** porque el EDP por sí solo permitiría soluciones inaceptables. Un usuario de HPC que ve su trabajo tardar 40 % más no se consuela con que el EDP mejoró. El objetivo general del trabajo dice literalmente "sin inducir una penalización severa en su rendimiento global"; **slowdown_limit** es la operacionalización numérica de esa frase.
- **Si es demasiado alto:** la tabla elige frecuencias agresivamente bajas y el agente se vuelve inaceptable en la práctica.
- **Si es demasiado bajo:** la tabla no puede alejarse de la referencia y no hay ahorro que demostrar.

delegated_cpus — el dominio de autoridad

- **Qué representa:** los núcleos sobre los que el daemon tiene permiso de escritura. **Fuera de esta lista no escribe jamás.**
- **Valor en **paccaA100**: 6 núcleos**, un hilo por núcleo físico [medido].
- **Por qué es un no-negociable:** **paccaA100** es un clúster compartido. En muchos procesadores la frecuencia se controla por grupos de núcleos, no por núcleo; si el dominio real de control excediera lo delegado, bajar "mi" frecuencia bajaría también la de núcleos que están corriendo el trabajo de otra persona. Por eso el preflight lee **freqdomain_cpus**, **related_cpus** y **affected_cpus** y **bloquea el arranque** si el dominio real no está contenido en lo delegado.

f_ref_level_id — el nivel de referencia

- **Qué representa:** el nivel contra el cual se congeló **i_ridge_ref** y contra el cual se normaliza el EDP de la calibración.
- **Valor por defecto: F0** (la máxima) si hay permiso de escritura; la frecuencia nativa del nodo si no.
- **Por qué importa declararlo:** porque **EDP_norm** y **slowdown** son cocientes contra este nivel. Reportar una mejora del 21 % sin decir contra qué referencia es no reportar nada.

El modelo serializado

- **Qué representa:** la función que va del vector de features a **p**.
- **De dónde sale:** Fase 2, seleccionado entre candidatos por el mejor compromiso entre desempeño predictivo y latencia de inferencia.
- **Si es demasiado complejo:** la latencia de inferencia empieza a competir con la época de 10 ms, y el cuarto objetivo específico —determinar si el ahorro compensa el sobrecosto— se responde en contra.
- **Si es demasiado simple:** no aporta nada sobre una regla de umbrales fijos, y el segundo objetivo específico pierde sentido.

4.3 Variables *trace*

Todas se observan; ninguna se fija. Pero no todas cumplen el mismo papel: solo un subconjunto entra al vector que ve el clasificador (componente ③); el resto sirve para verificar la actuación, calcular energía para el análisis de EDP, o alimentar las reglas de

precedencia que se evalúan **antes** de llegar al modelo. Confundir las dos columnas es fácil porque todas se leen del mismo hardware con la misma cadencia — por eso la tabla las separa de forma explícita.

Variable	Qué representa físicamente	Fuente exacta	Cadencia	¿Entra al modelo?
<code>ipc</code>	Instrucciones retiradas por ciclo. Cuánto trabajo útil produce el núcleo por unidad de tiempo de reloj. Bajo <code>ipc</code> con alto <code>mpki</code> es la firma de espera por memoria.	<code>perf_event_open</code>	1 ms → agregada	Sí
<code>mpki</code>	Fallos de caché por cada mil instrucciones. Presión de memoria normalizada por trabajo — a diferencia del conteo absoluto, permite comparar fases de distinta duración.	<code>perf_event_open</code>	1 ms → agregada	Sí
<code>llc_miss_rate</code>	Fracción de referencias a caché que no se resuelven en la jerarquía y deben ir a memoria.	<code>perf_event_open</code>	1 ms → agregada	Sí
<code>stall_backend_ratio</code>	Fracción de ciclos en que el pipeline no avanza esperando recursos o datos. La señal más directa de que el cuello de botella no es aritmético.	<code>perf_event_open</code>	1 ms → agregada	Sí
<code>ips</code>	Instrucciones por segundo. A diferencia de <code>ipc</code> , escala con la frecuencia — es la que permite ver si un cambio de frecuencia se tradujo en trabajo.	derivada	por época	Sí
<code>ipc_relative</code> , <code>mpki_relative</code> , <code>miss_rate_relative</code>	Las anteriores divididas por el percentil 95 de la	derivadas	por época	Sí

Variable	Qué representa físicamente	Fuente exacta	Cadencia	¿Entra al modelo?
	calibración de referencia del nodo.			
gpu_util_pct, gpu_mem_util_pct	Utilización de multiprocesadores y de memoria. Ambas: una carga con mucho tráfico de memoria y poco uso de SM parece ociosa si solo se mira la primera.	<code>nvmlDeviceGetUtilizationRates()</code>	100 ms	Sí (modelo GPU)
gpu_power_mw	Potencia instantánea de GPU.	<code>nvmlDeviceGetPowerUsage()</code>	100 ms	Sí (modelo GPU)
running_ratio	<code>time_running / time_enabled</code> . Detecta multiplexación de contadores.	<code>perf_event_open</code>	por época	No — gate de calidad (componente ①), decide si se infiere, no qué se infiere
pkg_delta_uj, dram_delta_uj	Energía consumida en el intervalo, en microjulios.	<code>/sys/class/powercap/intel-rapl/intel-rapl:0/energy_uj</code>	1 ms, por diferencias	No — solo para el cálculo de EDP en la Fase 4, nunca visto por el modelo
gpu_sm_clock_mhz	Reloj de SM vigente.	<code>nvmlDeviceGetClockInfo()</code>	100 ms	No — es la relectura que confirma una aplicación (componente ⑥), no una entrada de inferencia
gpu_energy_mj	Energía acumulada de GPU, por diferencias.	<code>nvmlDeviceGetTotalEnergyConsumption()</code>	100 ms	No — mismo papel que <code>pkg_delta_uj</code> , insumo de EDP
gpu_temperature_c	Temperatura.	<code>nvmlDeviceGetTemperature()</code>	100 ms	No — entrada de la verificación de salud (regla de precedencia P-1), no del modelo
freq_khz_observed	Frecuencia realmente vigente en un núcleo delegado.	<code>scaling_cur_freq</code>	tras cada escritura	No — relectura del actuador

Variable	Qué representa físicamente	Fuente exacta	Cadencia	¿Entra al modelo?
<code>gpu_busy</code>	Señal derivada: la GPU está haciendo trabajo activo.	derivada de utilización y potencia	500 ms	No — dispara la regla de precedencia P-2, que se evalúa antes de llamar al modelo y cuando se activa lo reemplaza
<code>residence_elapsed_ms</code>	Tiempo desde la última transición aplicada en este dominio.	<code>CLOCK_MONOTONIC</code>	por época	No — gate de la máquina de estados (componente ⑦), posterior a la inferencia

Lo que deliberadamente no aparece en esta tabla: `flops`, `bytes_moved`, `operational_intensity`. No es una omisión — es una condición de diseño con tres razones, y vale la pena tenerlas presentes porque son la respuesta a la pregunta más natural que se le puede ocurrir a un lector que conoce la Fase 1: *si la etiqueta de entrenamiento sale de comparar intensidad operacional contra el ridge, ¿por qué el daemon no hace lo mismo en vivo?*

- Porque sería fuga de la etiqueta.** `phase_label_train` se calcula exactamente así: `operational_intensity` contra `i_ridge`. Si esa misma intensidad fuera una entrada del modelo, el clasificador no necesitaría aprender nada de `ipc/mpki/stall_backend_ratio` — le bastaría con repetir la regla de la que salió su propia etiqueta. No sería un modelo entrenado, sería el umbral de Roofline con un paso intermedio inútil.
- Porque en GPU es físicamente imposible en producción.** Medir intensidad operacional requiere `ncu` (Nsight Compute), instrumentación de perfilado que el diseño de la política ya excluyó del daemon por decisión explícita — se usa una única vez, offline, en Fase 1, para etiquetar el dataset, y nunca vuelve a ejecutarse después. El plan aprobado fija que en producción GPU solo entrega NVML: utilización, memoria, potencia. Si CPU pudiera usar intensidad operacional y GPU no, los dos clasificadores dejarían de seguir la misma metodología.
- Porque sin esta restricción no haría falta ningún modelo.** Si la intensidad operacional estuviera disponible barata y en vivo, se compararía directo contra `i_ridge_ref` y se acabó — es la regla de Roofline sin aprendizaje de por medio. El clasificador existe para aproximar esa comparación **desde señales que sí se pueden leer en cada época**, sin necesitar FLOPs ni bytes movidos en tiempo real.

`operational_intensity` e `i_ridge` no desaparecen del proyecto: siguen siendo indispensables, pero exclusivamente del lado *oracle* (sección 4.2) — para calcular la etiqueta de entrenamiento en Fase 1, y para congelar `i_ridge_ref` en la política contra el cual se decidió qué frecuencia corresponde a cada clase (sección 5). El daemon, en cada época, nunca los calcula ni necesita conocerlos: solo le pasa el vector de la tabla de arriba — marcado "Sí" — al modelo ya entrenado, y lee la probabilidad que devuelve.

Dos notas que no son detalles.

Sobre las features relativas: un `ipc` de 1,85 no significa nada por sí solo — significa mucho o poco según de qué procesador se hable. Al normalizar contra el percentil 95 observado en la calibración del propio nodo, el modelo recibe "qué tan alto es esto **para esta máquina**" en lugar de un número absoluto. Es lo que le da alguna posibilidad de transferirse a otro nodo.

Sobre la energía de GPU: se obtiene del contador acumulado, **nunca integrando la potencia**. El valor de potencia de NVML es un indicador filtrado y con retardo; integrarlo a través de fronteras de fase acumula el error del filtro justo donde más importa — en las transiciones, que es donde el daemon actúa. Nótese que esto es sobre el uso de `gpu_power_mw/gpu_energy_mj` **para el cálculo de EDP** (columna "no" de la tabla); `gpu_power_mw` instantáneo sí es, por separado, una de las dos entradas del modelo de GPU junto con la utilización — son dos usos distintos de dos lecturas relacionadas pero no intercambiables.

5. Cómo se llena la tabla: ejemplo numérico completo

Este es el procedimiento que produce `HIGH` y `LOW`. Todo ocurre **offline**, después de la campaña.

5.1 Los niveles de frecuencia

Los cinco niveles se definen como porcentajes del rango real del nodo, no como valores absolutos:

Nivel	% del rango [f_min, f_max]
F0	100 % (máxima)
F1	75 %
F2	50 %
F3	25 %
F4	0 % (mínima)

Con un rango ilustrativo de 800–3600 MHz, eso da $F0 = 3600$, $F1 = 2900$, $F2 = 2200$, $F3 = 1500$, $F4 = 800$ MHz [ilustrativo].

5.2 Los datos que produce la campaña

Para cada clase, cada nivel y cada repetición se mide tiempo y energía. Agregando por clase [ilustrativo, magnitudes de potencia ancladas al rango 114–141 W medido en las campañas reales]:

Clase **compute_bound**:

Nivel	T (s)	P media (W)	E (J)	EDP (J·s)	EDP_norm	slowdown
F0	100	141	14 100	1 410 000	1,000	0 %
F1	104	124	12 896	1 341 184	0,951	4 %
F2	116	106	12 296	1 426 336	1,012	16 %
F3	134	93	12 462	1 669 908	1,184	34 %
F4	159	83	13 197	2 098 323	1,488	59 %

Clase **memory_bound**:

Nivel	T (s)	P media (W)	E (J)	EDP (J·s)	EDP_norm	slowdown
F0	100	130	13 000	1 300 000	1,000	0 %
F1	101	112	11 312	1 142 512	0,879	1 %
F2	103	97	9 991	1 029 073	0,792	3 %
F3	106	85	9 010	955 060	0,735	6 %
F4	118	76	8 968	1 058 224	0,814	18 %

5.3 Qué enseña este ejemplo — tres lecciones

Lección 1: **compute_bound NO significa frecuencia máxima.** El mínimo de EDP de la clase de cómputo está en **F1** (0,951), no en **F0**. Bajar un escalón desde la máxima ahorra energía superlinealmente mientras el tiempo solo crece 4 %. La heurística intuitiva **compute** → **fmax** habría elegido **F0** y habría perdido un 5 % de EDP. **Esto es exactamente por qué la tabla se mide y no se escribe a mano.**

Lección 2: **memory_bound NO significa frecuencia mínima — la curva tiene un mínimo interior.** Mírese la columna de energía de la clase de memoria: de **F3** a **F4** la energía baja de 9 010 a 8 968 J, apenas un 0,5 %, mientras el tiempo sube de 106 a 118 s, un 11 %. **La energía dejó de bajar.** La razón es física: la potencia estática y el consumo del uncore y de la memoria no bajan cuando baja el reloj del núcleo, así que existe un piso que se paga por unidad de tiempo pase lo que pase. Si se baja demasiado la frecuencia, ese piso se paga durante más tiempo del que se ahorra en potencia dinámica. Es el fenómeno de *race to idle*: a veces terminar rápido y quedarse quieto consume menos que ir despacio. El EDP lo refleja: **F4** (0,814) es **peor** que **F3** (0,735).

Lección 3: la restricción de rendimiento manda sobre el EDP. El mínimo de EDP de la clase de memoria está en **F3** (0,735), pero su degradación es del 6 %, por encima del límite del 5 %. **F3** queda descartado. El nivel elegido es **F2**: **EDP_norm** = 0,792,

degradación 3 %. Se sacrifica algo de ahorro para respetar la promesa de "sin penalización severa del rendimiento" — que es lo que dice el objetivo general del trabajo.

5.4 El resultado

```
HIGH_CPU = F1 (2900 MHz)  EDP_norm 0,951  slowdown 4 %
LOW_CPU  = F2 (2200 MHz)  EDP_norm 0,792  slowdown 3 %
```

La formulación general:

```
f*(dominio, clase) = argmin_f  mediana_w( EDP_norm(w, dominio, f) )
                      sujeto a  slowdown_máximo(f) ≤ slowdown_limit
                      y          fracción_de_workloads_mejorados(f) ≥ umbral
```

5.5 Tres reglas estadísticas que no se pueden saltar

(a) Normalizar por workload es obligatorio. Sin normalizar, un kernel largo domina la estadística por la magnitud absoluta de su EDP, no por su comportamiento. Al dividir cada workload por su propio EDP en el nivel de referencia, todos entran a la mediana con el mismo peso y $F_{ref} = 1,000$ para todos.

(b) No usar percentil 95 con $n = 3$. Con tres repeticiones por celda, el P95 empírico es indistinguible del máximo. El criterio de degradación usa el **máximo observado** — conservador y honesto — y el intervalo de confianza se construye por *bootstrap* sobre la mediana **entre workloads** (6–7 unidades), que sí tiene material suficiente.

(c) Ante empates estadísticos, elegir el conservador. Si dos niveles tienen intervalos de confianza solapados, se elige el de menor degradación. Presentar una diferencia dentro del ruido como una decisión es el error metodológico más fácil de cometer aquí.

5.6 Dos resultados que hay que aceptar si aparecen

- **HIGH == LOW.** Los datos dicen que una única frecuencia domina en ambas clases. Es un resultado científico válido: significa que en este nodo, con estas cargas, la política dinámica no aporta sobre la mejor estática. Hay que reportarlo, no forzar una diferencia.
- **La mejor estática le gana al oráculo.** Indica fases más cortas que el umbral de ~140 ms, costo de conmutación alto, o granularidad binaria insuficiente. También es publicable, y el diseño experimental está construido para poder distinguir esos tres casos.

6. Por qué esta política minimiza el EDP: el argumento físico

Este es el núcleo científico del trabajo y conviene tenerlo escrito con precisión.

El Producto Energía–Retardo es $EDP = E \times T$. La energía es la integral de la potencia en el tiempo. En un procesador CMOS la potencia tiene dos componentes:

```
P_total = P_dinámica + P_estática
          ~ C · V² · f      no depende de f
```

Y como el voltaje debe subir para sostener frecuencias más altas, la dependencia efectiva de $P_{dinámica}$ respecto de la frecuencia es **marcadamente superlineal**. Bajar la frecuencia ahorra potencia rápido.

La pregunta que decide todo es: **¿cuánto crece T al bajar la frecuencia?** Y la respuesta depende del régimen — que es precisamente lo que el modelo Roofline formaliza y lo que el clasificador detecta:

En régimen limitado por cómputo, el camino crítico son las unidades funcionales del núcleo. Reducir el reloj a la mitad aproximadamente duplica el tiempo:

$T \sim 1/f \rightarrow$ E baja, pero T sube casi en la misma proporción
el producto $E \times T$ empeora

En el ejemplo de la sección 5, de F1 a F2 (una caída del 24 % en frecuencia) el tiempo sube de 104 a 116 s, un 12 %, y el EDP empeora de 0,951 a 1,012.

En régimen limitado por memoria, el camino crítico es el subsistema de memoria: latencia de DRAM y ancho de banda del controlador, que pertenecen a **un dominio de reloj distinto y no escalan con la frecuencia del núcleo**. Reducir el reloj del núcleo hace que este espere más ciclos por el mismo dato, pero el tiempo de pared casi no cambia porque el tiempo ya lo estaba fijando la memoria:

$T \approx \text{constante} \rightarrow$ E cae superlinealmente, T apenas se mueve
el producto $E \times T$ mejora

En el ejemplo, de F0 a F2 (una caída del 39 % en frecuencia) el tiempo sube de 100 a 103 s, apenas un 3 %, mientras la energía cae de 13 000 a 9 991 J, un 23 %. El EDP mejora un 21 %.

Esta asimetría es todo el trabajo. Y es la razón por la que el proyecto invirtió en calibración Roofline en lugar de usar utilización: **la utilización no distingue estos dos casos y el Roofline sí**. Un núcleo al 100 % esperando datos de DRAM y un núcleo al 100 % multiplicando se ven idénticos desde `top`, y radicalmente distintos desde `mpki` y `stall_backend_ratio`.

Contra qué se compara. Los gobernadores nativos de Linux reaccionan a carga y utilización:

Linux: utilización $\rightarrow$ frecuencia
Hyperion: comportamiento microarquitectónico $\rightarrow$ clase $\rightarrow$ frecuencia calibrada por EDP

Que la información adicional produzca efectivamente una mejora es **la hipótesis que la Fase 4 debe evaluar**, no algo que este documento afirme.

7. Traza completa de ejecución

Catorce épocas consecutivas de CPU (10 ms cada una, 140 ms de ejecución) que atraviesan un cambio de fase. Configuración: `tau = 0,15, 3 de 4`, `min_residence = 100 ms`, `HIGH = F1 (2900 MHz)`, `LOW = F2 (2200 MHz)`. Estado inicial: F1 aplicado en `t = 0`. [ilustrativo]

#	t (ms)	ipc	mpki	stall	p	Candidato	Historial	Decisión	f (MHz)	Motivo
1	0	1,86	2,1	0,19	0,93	HIGH	H	NO_CHANGE	2900	already_there
2	10	1,91	1,8	0,17	0,95	HIGH	HH	NO_CHANGE	2900	already_there
3	20	1,84	2,4	0,21	0,91	HIGH	HHH	NO_CHANGE	2900	already_there
4	30	0,62	21,0	0,58	0,28	LOW	HHHL	NO_CHANGE	2900	unstable (1 de 4)
5	40	1,88	2,0	0,18	0,94	HIGH	HHLH	NO_CHANGE	2900	already_there
6	50	0,44	31,2	0,71	0,19	LOW	HLHL	NO_CHANGE	2900	unstable (2 de 4)
7	60	0,39	34,8	0,76	0,14	LOW	LHLL	NO_CHANGE	2900	unstable (3 de 4) → ver nota
8	70	0,37	36,1	0,78	0,11	LOW	HLLL	NO_CHANGE	2900	min_residence (70/100)
9	80	0,38	35,4	0,77	0,12	LOW	LLLL	NO_CHANGE	2900	min_residence (80/100)
10	90	0,36	37,0	0,79	0,10	LOW	LLLL	NO_CHANGE	2900	min_residence (90/100)
11	100	0,35	37,8	0,80	0,09	LOW	LLLL	SWITCH_OK	2200	releído 2200 ✓

#	t (ms)	ipc	mpki	stall	p	Candidato	Historial	Decisión	f (MHz)	Motivo
12	110	0,52	14,2	0,44	0,52	—	LLLL	NO_CHANGE	2200	undecided_band
13	120	1,74	1,9	0,16	—	—	LLLL	FLOOR	800	gpu_busy (sin inferencia)
14	130	—	—	—	—	—	LLLL	NO_CHANGE	800	telemetry_invalid (running_ratio 0,62)

Nota sobre la época 7: el historial **LHLL** tiene 3 de 4 **LOW**, así que el filtro de estabilidad **sí se satisface** y el candidato es aceptado. Lo que bloquea la acción a partir de aquí ya no es la estabilidad sino la residencia mínima — por eso el motivo de la época 8 cambia a **min_residence**. La transición del motivo de **unstable** a **min_residence** es la señal de que la política pasó de "no me convence la señal" a "me convence, pero todavía no toca".

Qué demuestra cada tramo

Épocas 1–3 — régimen de cómputo estable. **ipc** alto, **mpki** bajo, **stall** bajo. El modelo está muy seguro (**p** ≈ **0,93**). La política no hace nada porque **ya está en el estado correcto**: **already_there** es el motivo más frecuente en una ejecución sana, y eso es exactamente lo que se busca.

Época 4 — el filtro de estabilidad hace su trabajo. Una única época con perfil de memoria (**ipc** 0,62, **mpki** 21) y confianza alta (**p** = **0,28**). **La banda de indecisión no la habría atrapado** — 0,28 está muy por debajo de 0,35. Lo que la atrapa es el filtro temporal: 1 de las últimas 4. Y la época 5 confirma que era una ventana atípica. Sin este filtro, aquí habría habido una transición innecesaria, seguida de otra de vuelta.

Épocas 6–7 — la fase real empieza. **mpki** sube a 31 y 35, **stall** a 0,71 y 0,76, **p** cae a 0,19 y 0,14. La señal es persistente **y** el modelo está seguro. En la época 7 el filtro se satisface. **Latencia de detección: 3 épocas = 30 ms** desde el comienzo real de la fase.

Épocas 8–10 — la residencia mínima retiene la acción. La política ya decidió; el hardware todavía no ha amortizado la transición anterior. Se registran tres **NO_CHANGE** consecutivos con el tiempo restante anotado. **Este es el componente que más NO_CHANGE produce en una ejecución real, y es el que protege el ahorro.**

Época 11 — la transición. Residencia cumplida (100 ms), se escribe 2200 MHz, se relee 2200 MHz, confirmado. **Latencia total desde el inicio de la fase hasta la acción: 60 ms** (30 de detección + 30 de espera por residencia). El temporizador de residencia se reinicia aquí.

Época 12 — la banda de indecisión. **p** = **0,52**: el modelo está prácticamente indeciso, y las métricas lo respaldan (**ipc** 0,52, **mpki** 14,2 — valores intermedios). Es el perfil de una carga mixta como **npb_sp**. **NO_CHANGE**. **Con tau = 0,05 esta época habría producido un candidato HIGH y, de repetirse, una transición de vuelta a 2900 MHz sobre una confianza del 52 %.**

Época 13 — la regla de precedencia. La GPU está activa. Nótese que **ipc** volvió a 1,74 y **mpki** a 1,9: **el perfil es exactamente el de una carga de cómputo**, y el modelo habría dicho **compute_bound** con alta confianza. Pero es espera activa por la GPU: el CPU está girando en vacío. La política **no llama al clasificador** y lleva la frecuencia al piso. Sin esta regla, aquí se habría subido la frecuencia justo cuando el CPU no hace trabajo útil — el peor caso posible para el EDP.

Época 14 — telemetría inválida. **running_ratio** = **0,62**: contadores multiplexados. No hay features utilizables, no hay inferencia, **NO_CHANGE**. La frecuencia se queda donde está.

Cuentas de la traza

épocas totales	14	
llamadas al clasificador	12	(13 y 14 no llegaron a inferir)
transiciones aplicadas	1	
transiciones evitadas	4	(1 por ruido, 3 por residencia)
transiciones evitadas por banda	1	
correcciones por precedencia	1	
épocas descartadas por calidad	1	

Una transición en 140 ms. Ese es el punto: la política está construida para actuar poco y bien. Cada **NO_CHANGE** de la tabla anterior es una transición que no se pagó.

8. Los cuatro casos difíciles

8.1 Carga genuinamente mixta

El caso. `npb_sp` reparte sus ventanas 58,2 % / 41,8 % entre clases, de forma estable entre dos tamaños de problema [medido]. No es error de medición: es una carga que alterna a una granularidad más fina que la época.

Qué hace la política. La banda de indecisión mantiene el estado actual. La política no intenta seguir una señal que oscila más rápido de lo que puede actuar.

Por qué es correcto. Porque el alternativo —conmutar— pagaría 10 ms de transición por cada cambio sobre una señal que no llega al 65 % de confianza. La política prefiere quedarse en un estado subóptimo y estable antes que perseguir un óptimo inalcanzable.

Qué se declara en el libro. Que la granularidad binaria es insuficiente para cargas mixtas, y que la política lo reconoce explícitamente en lugar de fingir que decide.

8.2 Bistabilidad del `ridge` — por qué `i_ridge_ref` está congelado

Aclaración antes del ejemplo, para que no se lea como una contradicción de la sección 4.3: el daemon **no calcula `operational_intensity` en vivo** — eso quedó establecido como regla de diseño (§4.3): sería fuga de la etiqueta, es físicamente imposible en GPU en producción, y volvería innecesario al propio clasificador. Lo que sigue es un experimento mental sobre el procedimiento **offline** que produce `i_ridge_ref` y la tabla de frecuencias (secciones 4.2 y 5): muestra qué pasaría si ese procedimiento offline usara, en cada nivel de frecuencia, el ridge de ese *mismo* nivel en lugar de un ridge de referencia fijo. La conclusión —que hay que congelar el ridge contra un único nivel de referencia— es precisamente la razón por la que `i_ridge_ref` es una constante *oracle* y nunca una cantidad que el daemon vuelva a tocar en tiempo de ejecución.

El caso, con números reales. El ridge de GPU para precisión simple es $10\,178,2\text{ GFLOP/s} \div 1\,399\text{ GB/s} = 7,28\text{ FLOP/byte}$ [medido]. El kernel `rodinia_hotspot` tiene una intensidad operacional medida con Nsight Compute de **5,03 FLOP/byte** [medido].

A reloj completo: $5,03 < 7,28 \rightarrow \text{memory_bound}$.

Ahora supóngase que la política, correctamente, aplica **LOW** y el reloj de SM baja a la mitad. El pico de cómputo escala con el reloj; el ancho de banda de memoria **no**, porque el reloj de memoria es un dominio aparte que este proyecto no controla — de hecho el A100 del nodo solo soporta un único valor de reloj de memoria [medido]. Entonces:

```
P_pico(medio reloj) ≈ 5 089 GFLOP/s
BW_pico              = 1 399 GB/s   (sin cambio)
i_ridge(medio reloj) ≈ 3,64 FLOP/byte
```

Y ahora: $5,03 > 3,64 \rightarrow \text{compute_bound}$. El mismo kernel, sin haber cambiado en nada, cruzó de clase solo porque la política actuó.

El lazo cerrado:

```
hotspot clasificado memory_bound (5,03 < 7,28)
  → política aplica LOW
    → baja el reloj de SM
      → baja P_pico
        → baja i_ridge a 3,64
          → hotspot ahora es compute_bound (5,03 > 3,64)
            → política aplica HIGH
              → sube el reloj
                → sube i_ridge a 7,28
                  → hotspot vuelve a memory_bound ...
```

Por qué la histéresis NO lo arregla. Esto no es ruido: es un ciclo límite determinista. Ni la banda de indecisión ni el filtro **3 de 4** ni la residencia mínima lo eliminan — **solo alargan su periodo**. Con residencia de 100 ms, en lugar de oscilar cada época, oscila cada 100 ms; indefinidamente; pagando una transición cada vez.

La solución. La etiqueta que cierra el lazo de control se calcula contra `i_ridge_ref`, un valor **constante congelado en el archivo de política**, no contra el ridge del estado actual. La justificación no es solo práctica sino conceptual: **la etiqueta debe ser una propiedad de la carga, no del actuador**. Si depende del actuador, el sistema está clasificando su propia decisión anterior en lugar de clasificar la aplicación.

Lo que no se pierde. La etiqueta dependiente de frecuencia se conserva y se calcula igual, pero **para el análisis del capítulo de resultados**, no para el control. Que el ridge dependa de la frecuencia es un hallazgo real de la caracterización de este proyecto y merece su sección; simplemente no es la señal que cierra el lazo.

8.3 Espera activa por GPU

Desarrollado en el componente ② y en la época 13 de la traza. Resumen: la señal es genuinamente engañosa, ningún entrenamiento la arregla porque los vectores de features son idénticos, y la respuesta es una regla de precedencia que se evalúa **antes** de consultar al modelo.

8.4 Telemetría degradada

El caso. Multiplexación de contadores, denominador cero, muestras faltantes.

Qué hace la política. `NO_CHANGE`, con el motivo registrado. **No se imputa, no se rellena, no se "arregla" el dato.**

Por qué. Porque en este pipeline los errores de datos no se manifiestan como fallos ruidosos sino como resultados plausibles y equivocados. La regla del proyecto es explícita: denominador cero produce un valor no numérico y un estado de calidad, nunca un cero silencioso. Una época degradada que se "arregla" con un valor imputado es una época que decide sobre un número inventado, y no hay forma de distinguirla después en el log.

9. Qué NO decide la política

Delimitarlo protege las conclusiones:

- **No decide cuántos núcleos usa la aplicación.** Eso lo fija el manifiesto, no el daemon.
- **No decide límites de potencia.** El *power capping* está fuera del alcance del plan aprobado.
- **No decide sobre el reloj de memoria.** En el A100 del nodo solo hay un valor soportado [medido]; el espacio DVFS de GPU es unidimensional.
- **No decide sobre núcleos fuera de los delegados.** Nunca, bajo ninguna condición.
- **No predice cuánto durará la fase actual.** Reacciona a la fase observada; no anticipa su final. Predecir duración de fases sería otro proyecto.
- **No aprende durante la ejecución.** No hay exploración, ni actualización de la tabla, ni ajuste de umbrales en caliente. Todo lo oracle es constante.
- **No garantiza optimalidad.** Garantiza una decisión trazable y acotada bajo una restricción declarada de rendimiento. Que esa decisión mejore el EDP frente a los baselines es la hipótesis que la Fase 4 evalúa.

10. Resumen de una página

La política en una frase: *dado un régimen de ejecución inferido con confianza suficiente y sostenido en el tiempo, aplicar el nivel de frecuencia que la campaña de caracterización identificó como el de menor EDP para ese régimen, siempre que haya transcurrido el tiempo necesario para amortizar la transición anterior y el hardware esté en un estado conocido.*

Los ocho componentes:

#	Componente	Pregunta que responde	Salida si falla
①	Validación de telemetría	¿los datos sirven?	<code>NO_CHANGE</code>
②	Reglas de precedencia	¿hay algo que manda sobre el modelo?	<code>FAILSAFE</code> / <code>FLOOR</code>
③	Inferencia	¿qué régimen es?	—
④	Banda de indecisión	¿el modelo está seguro?	<code>NO_CHANGE</code>
⑤	Filtro de estabilidad	¿la señal persiste?	<code>NO_CHANGE</code>

#	Componente	Pregunta que responde	Salida si falla
⑥	Consulta a la tabla	¿qué frecuencia corresponde?	NO_CHANGE
⑦	Residencia mínima	¿ya se amortizó la anterior?	NO_CHANGE
⑧	Aplicación + relectura	¿se aplicó de verdad?	FAILSAFE

Las variables oracle y sus valores:

Variable	Valor	Origen
i_ridge_ref	7,28 / 3,36 FLOP/byte (GPU fp32/fp64)	medido, calibración del nodo
HIGH, LOW	null	pendiente de la campaña
tau	0,15	barrido offline sobre validación
n_of_m	3 de 4	compromiso latencia/robustez
min_residence_ms	100 (CPU) / 1000 (GPU)	$10 \times T_{transición}$
slowdown_limit	5 %	operacionaliza "sin penalización severa"
delegated_cpus	6 núcleos	asignación del nodo
f_ref_level_id	F0 o nativa	según permiso de escritura

Los números que definen el comportamiento temporal:

época de decisión CPU	10 ms	
latencia de detección de fase	30-40 ms	
residencia mínima	100 ms	
fase mínima aprovechable	≈140 ms	← limitación declarada

Lo que la política no puede hacer hoy: llenar HIGH y LOW con números reales. Eso depende de la campaña multi-frecuencia, que depende del permiso de escritura de frecuencia. Todo lo demás de este documento está definido, es implementable y es verificable sin ese permiso.